

PYTHON OOP — PRACTICAL CHEAT SHEET

Commands, syntax, examples, common patterns, and quick reference for beginners

1. Python Basics You Need for OOP

Variables & Data Types

```
name = "Alice"
age = 15
hp = 100
damage = 25
is_alive = True
```

Common types: int, float, str, bool, list, tuple, dict, set, None

Conditions

```
if hp > 0:
    print("Alive")
elif hp == 0:
    print("Dead")
else:
    print("Invalid HP")
```

Loops

```
for i in range(5):
    print(i)

for item in items:
    print(item)

while hp > 0:
    hp -= 10
```

Functions

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result) # 30
```

Useful Built-ins

Command	Purpose	Example
type()	Check a value's type	type(hero)
len()	Get length	len(players)
print()	Display output	print(hero.name)
str()	Convert to string	str(hero.hp)
int()	Convert to integer	int("10")
isinstance()	Check object type	isinstance(hero, Hero)

2. Class & Object

Define a Class

```
class Hero:
    pass
```

Class = blueprint. Object = a specific instance created from the class.

Create Objects

```
hero1 = Hero()
hero2 = Hero()
print(type(hero1))
```

Basic Class Example

```
class Hero:
    def __init__(self, name, hp, damage):
        self.name = name
        self.hp = hp
        self.damage = damage

hero = Hero("Archer", 100, 25)
```

Access & Change Attributes

```
print(hero.name)
print(hero.hp)
```

```
hero.hp = 80  
hero.damage += 5
```

Remember: `self.name` belongs to each object. `hero.name` accesses that object's value.

3. `__init__()` and `self`

`__init__()`

`__init__()` runs automatically when a new object is created and is normally used to initialize instance attributes.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student = Student("Anna", 15)
```

`self`

`self` means the current object/instance. It lets a method access that object's attributes and other methods.

```
class Hero:
    def show_info(self):
        print(self.name)
        print(self.hp)
```

Common mistake: Inside the class, use `self.name` for the current object's attribute.

4. Instance Attributes vs Class Attributes

Instance Attribute

```
class Hero:
    def __init__(self, name):
        self.name = name

hero1 = Hero("Warrior")
hero2 = Hero("Mage")
```

Class Attribute

```
class Hero:
    game = "Fantasy World"

hero1 = Hero()
hero2 = Hero()

print(hero1.game)
print(hero2.game)
```

Type	Belongs to	Example
Instance attribute	One object	<code>self.name</code>
Class attribute	The class	<code>Hero.game</code>

5. Methods

Instance Method

```
class Hero:
    def attack(self):
        print(self.name, "attacks!")

hero.attack()
```

Method with Parameters

```
def attack(self, target):
    print(self.name, "attacks", target.name)

hero.attack(enemy)
```

Method with return

```
def get_damage(self):
    return self.damage

damage = hero.get_damage()
```

<code>print()</code>	<code>return</code>
Displays a value	Sends a value back to the caller
Mainly for output/debugging	Can be stored or used in calculations

6. Attribute Operations

```
# Read
print(hero.hp)

# Update
hero.hp = 90

# Add
hero.level = 5

# Delete
del hero.level
```

hasattr / getattr / setattr

```
if hasattr(hero, "weapon"):
    print(hero.weapon)

weapon = getattr(hero, "weapon", "None")

setattr(hero, "level", 10)
```

7. Encapsulation

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance
        self.__pin = 1234
```

`_name` → internal/protected-by-convention. `__name` → name mangling. Python does not enforce private fields like some languages.

Controlled Access

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance
```

8. Inheritance

Basic Inheritance

```
class Hero:
    def attack(self):
        print("Hero attacks")

class Mage(Hero):
    pass

mage = Mage()
mage.attack()
```

super()

```
class Hero:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

class Mage(Hero):
    def __init__(self, name, hp, mana):
        super().__init__(name, hp)
        self.mana = mana
```

Method Overriding

```
class Hero:
    def attack(self):
        print("Basic attack")

class Mage(Hero):
    def attack(self):
        print("Cast fireball")
```

is-a example: Mage is a Hero. Use inheritance for a genuine specialization relationship.

9. Polymorphism

```
class Warrior:
    def attack(self):
        print("Sword attack")

class Mage:
    def attack(self):
        print("Magic attack")

class Archer:
    def attack(self):
        print("Arrow attack")

heroes = [Warrior(), Mage(), Archer()]

for hero in heroes:
    hero.attack()
```

Key idea: the same call `attack()` can produce different behavior depending on the object.

10. Composition

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()

car = Car()
car.start()
```

Composition: Car has an Engine. This is different from inheritance, where a child is a parent type.

11. Special / Dunder Methods

`__str__()`

```
class Hero:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

    def __str__(self):
        return f"{self.name} ({self.hp} HP)"

hero = Hero("Archer", 100)
print(hero)
```

Method	Purpose	Typical use
<code>__init__</code>	Initialize object	Create attributes
<code>__str__</code>	Readable string	<code>print(object)</code>
<code>__repr__</code>	Developer representation	Debugging
<code>__len__</code>	Define <code>len(object)</code>	<code>len(object)</code>
<code>__eq__</code>	Define <code>==</code> behavior	<code>object1 == object2</code>

12. `isinstance()` and `issubclass()`

```
hero = Mage("Merlin", 80, 100)

print(isinstance(hero, Mage)) # True
print(isinstance(hero, Hero)) # True

print(issubclass(Mage, Hero)) # True
```

13. Lists of Objects

Store Objects in a List

```
heroes = [  
    Hero("Warrior", 120, 30),  
    Hero("Mage", 80, 40),  
    Hero("Archer", 100, 25)  
]  
  
for hero in heroes:  
    print(hero.name, hero.hp)
```

Filter

```
alive_heroes = []  
  
for hero in heroes:  
    if hero.hp > 0:  
        alive_heroes.append(hero)
```

Sort

```
heroes.sort(key=lambda hero: hero.hp)  
  
for hero in heroes:  
    print(hero.name, hero.hp)
```

14. Complete Beginner OOP Example

```
class Hero:  
    def __init__(self, name, hero_type, hp, damage):  
        self.name = name  
        self.hero_type = hero_type  
        self.hp = hp  
        self.damage = damage  
  
    def introduce(self):  
        return f"{self.name} is a {self.hero_type}. HP: {self.hp}"  
  
    def attack(self, target):  
        target.hp -= self.damage  
        print(f"{self.name} attacks {target.name}!")  
  
    def is_alive(self):  
        return self.hp > 0  
  
warrior = Hero("Thor", "Warrior", 120, 30)  
archer = Hero("Robin", "Archer", 100, 25)  
  
print(warrior.introduce())  
archer.attack(warrior)  
print(warrior.hp)  
print(warrior.is_alive())
```

15. Quick Syntax Reference

Task	Syntax
Create class	class Hero:
Constructor	def __init__(self, ...):
Create object	hero = Hero(...)
Instance attribute	self.hp = 100
Read attribute	hero.hp
Change attribute	hero.hp = 80
Call method	hero.attack()
Return value	return value
Inheritance	class Mage(Hero):
Call parent	super().__init__(...)
Check type	isinstance(obj, ClassName)
Check inheritance	issubclass(Child, Parent)

16. Common Mistakes

Forgetting self

```
class Hero:
    def attack():    # Wrong
        print("Attack")

# Correct:
def attack(self):
    print("Attack")
```

Forgetting parentheses

```
hero.attack    # Refers to the method
hero.attack()  # Calls the method
```

Confusing print with return

```
def get_hp(self):
    return self.hp

hp = hero.get_hp()
```

Wrong relationship for inheritance

```
# "is-a" -> inheritance
class Mage(Hero):
    pass

# "has-a" -> composition
class Car:
    def __init__(self):
        self.engine = Engine()
```

17. OOP Vocabulary

Term	Simple meaning
Class	Blueprint/template
Object	Instance of a class
Attribute	Data/state stored on an object
Method	Function belonging to a class/object
Encapsulation	Keep data and behavior together; control access
Inheritance	Create a class based on another class
Override	Replace inherited method behavior
Polymorphism	Same interface/method, different behavior
Composition	Build an object using other objects

18. Final Practice

- ☐ Create a class from scratch.
- ☐ Use `__init__` with 3-4 attributes.
- ☐ Create several objects with different values.
- ☐ Write methods that read and modify attributes.
- ☐ Use parameters and return values.
- ☐ Create a parent and child class.
- ☐ Use `super()`.
- ☐ Override a method.
- ☐ Demonstrate polymorphism.
- ☐ Use composition for a has-a relationship.
- ☐ Use `__str__`.
- ☐ Store and loop through objects.

Golden rule: Type the examples yourself, change the values, intentionally break the code, and fix it. Practice is more important than memorizing syntax.